

AiProf Study Companion

AI Tools & Usage Technical Documentation

A Comprehensive, Transparent Disclosure of AI Assistance During Development and the Production AI Models Powering the Deployed Application.

Candidate / Developer: **Sathwik Bodakunta**

Submission Track: **Full Stack AI Engineer Intern**

Repository: `github.com/sathwik1821/Ai-Prof-Study-Companion`

Live Web App: `ai-prof-study-companion.vercel.app`

AI in Development: **Google Antigravity IDE (Agentic Assistant & Tooling)**

AI in Product: **Google Gemini Flash Lite & gemini-embedding-001**

Vector Subsystem: **PostgreSQL 16 pgvector (3072-dimensional cosine search)**

Date: **September 2026**

Transparent AI
Disclosure

Developer Review
Verification

Socratic RAG
Tutoring

Adaptive Quiz
Generation

Zero Exposed
Secrets

1. Overview & Separation of AI Roles

AiProf Study Companion is an enterprise-grade, full-stack educational platform built to transform unstructured academic documents into grounded, interactive learning journeys. Because Artificial Intelligence played a role in both the *engineering process* and the *resulting user experience*, this technical document formally and transparently delineates the two distinct categories of AI involvement:

Mandatory Architectural Separation of Concerns:

1. **AI Used to Build the Product (Category A):** Autonomous developer agents, IDE coding assistants, refactoring engines, browser subagents, and automated validation tooling utilized by the developer (Sathwik Bodakunta) during software engineering.
2. **AI Embedded in the Final Product (Category B):** Production inference models (Google Gemini Flash Lite, Gemini Embeddings) and vector database infrastructure (PostgreSQL pgvector) that execute inside the live deployed application to deliver tutoring, adaptive quizzing, and document RAG to end-users.

CATEGORY A: AI USED TO BUILD THE PRODUCT

Target Audience: Candidate / Developer (Sathwik Bodakunta)

Primary Tooling: Google Antigravity IDE (Agentic Assistant)

Key Applications:

- Spring Boot 3.4 & React 19 boilerplate generation
- Flyway SQL schema migration drafting (V1-V7)
- React hook order debugging (Error #310 resolution)
- Autonomous headless browser testing & DOM auditing

Human Role: 100% human architectural review & verification

CATEGORY B: AI EMBEDDED IN FINAL PRODUCT

Target Audience: End-Users, University Students, Evaluator

Runtime Models: Gemini Flash Lite • gemini-embedding-001

Production Features:

- Socratic AI Tutor with clickable document citations
- RAG document semantic chunking & 3072d vector search
- Adaptive multi-format diagnostic quiz generation
- Cognitive concept mastery & prescriptive recommendations

Runtime State: Integrated in live backend code & database

Figure 1.1: Complete Decoupling of AI Development Tools vs. Production AI Functionality

2. AI Used to Build the Product (Category A)

During the implementation of the AiProf platform, specialized AI-assisted development tools were utilized to accelerate boilerplate authoring, automate cross-file refactoring, perform root-cause debugging on compilation and runtime errors, and autonomously audit production deployments.

AI Tool / System	Category	Development Use	Detailed Description of Assistance
Google Antigravity IDE Agent	Development Agent / Coding Assistant	Full-Stack Implementation & Refactoring	Assisted in scaffolding Spring Boot 3.4.3 backend components, JPA entities, repositories, and React 19 UI layouts; performed non-contiguous edits across multi-file refactoring cycles.
Antigravity Browser Subagent	AI Debugging & Testing Tool	Autonomous Web Verification & Telemetry	Programmatically navigated the live Vercel production deployment, captured headless screenshots, read the active DOM, and captured console logs to detect runtime rendering crashes.
Gemini Technical Reasoning	Problem Solving & Architecture	Flyway Migration & Schema Alignment	Formulated version-controlled SQL migrations (<code>V1__init_extensions.sql</code> to <code>V7__email_verification_otp.sql</code>), ensuring vector column alignment and foreign key cascading rules.
Automated CLI Test Runners	Debugging & Compilation Agent	Build Integrity Validation	Monitored background execution of Maven test compilation (<code>mvnw test-compile</code>) and Vite production bundling (<code>npm run build</code>) after each incremental architectural change.

2.1 Major Development Tasks Accelerated by AI

- RAG Pipeline Scaffolding:** Assisted in writing the Apache Tika file ingestion logic, sliding-window chunking algorithms (400 tokens / 50 overlap), and Spring Data JPA native SQL queries calculating cosine distance (`<=>`) over `VECTOR(3072)` columns.
- Dual Authentication Integration:** Supported implementation of the Google Identity Services button integration, backend ID token validation via Google OAuth tokeninfo endpoints, and fallback email OTP generation.
- React Hook Order Debugging (Error #310):** When the dashboard threw a minified React Error #310 in production, the AI assistant identified that `useMemo` was called conditionally below an early `if (loading) return skeleton` check, and refactored the component to restore deterministic hook execution order.
- Cloud SMTP Firewall Mitigation:** Diagnosed that Render's Free tier firewall blocks outbound TCP ports 25, 465, and 587, and implemented the resilient Resend HTTP REST API (port 443) transport with auto-fill preview code fallbacks in `EmailService.java` and `OtpVerifyPage.jsx`.

3. Developer Contribution & Engineering Review

While AI tooling provided substantial acceleration, **the human developer maintained complete architectural ownership, technical judgment, and verification responsibility** across all phases of development. AI tools did not independently build the product; every artifact was subjected to developer review, adaptation, and testing.

Key Human Engineering Responsibilities Executed by Developer (Sathwik Bodakunta):

- **Requirements Interpretation & Scope Definition:** Defined core features including the Socratic tutoring pedagogy, the Empty Materials Gate, concept mastery algorithms, and sidebar navigation restructuring.
- **Architectural Decision-Making:** Selected PostgreSQL `pgvector` over external vector databases (e.g. Pinecone) to eliminate distributed synchronization latency and preserve ACID relational integrity.
- **Code Inspection & Modification:** Reviewed all generated DTOs, entity relationships, security filters, and React hooks. Rejected hardcoded test concepts in favor of dynamic database-backed mastery tracking.
- **Security & Secret Sanitization:** Audited `application.yml` and `.env.example` to scrub real Google App Passwords, API keys, and database credentials before committing to public GitHub.
- **Infrastructure & Deployment Management:** Configured Google Cloud Console OAuth 2.0 authorized JavaScript origins, Neon serverless PostgreSQL connection strings, Vercel SPA routing rules, and Render web service environment variables.
- **Functional & User Flow Validation:** Verified authentication flows, document uploads, vector ingestion, and quiz attempts through manual and browser-driven testing.

4. AI Used by the Final Product (Category B)

The production application deployed at `ai-prof-study-companion.vercel.app` contains five integrated AI-powered capabilities that execute in real time to serve end-user academic needs:

Product Feature	Educational Purpose	Runtime AI Model / Engine	User Interaction / Output
Socratic AI Tutor	Interactive inquiry-based learning that guides students through questioning rather than direct answers.	Google Gemini Flash Lite (<code>gemini-flash-lite-latest</code>)	Conversational dialogue with markdown support and clickable chunk citations: <code>[Citation: doc (chunk #)]</code> .
RAG Semantic Document Search	Retrieves the most semantically relevant text passages from uploaded course materials.	Google Gemini Embedding (<code>gemini-embedding-001</code>) + pgvector	Under-the-hood vector generation (3072 dimensions) and cosine distance nearest-neighbor queries (<code><=></code>).
Adaptive Quiz Synthesis	Synthesizes multi-format diagnostic assessments derived strictly from course documents.	Google Gemini Flash Lite (<code>gemini-flash-lite-latest</code>)	Multiple-choice, true/false, and short-answer quizzes with answer rationales and concept tags.
Concept Mastery Assessment	Quantifies cognitive retention (\$0% - 100%\$) per academic concept and diagnoses gaps.	Algorithmic scoring + AI concept categorization	Dynamic mastery radar charts, horizontal retention bars, and weak-concept alerts on Analytics.
Targeted Mastery Recommendations	Prescribes high-priority learning actions based on the student's weakest mastered concepts.	Heuristic AI recommendation engine in <code>MasteryService</code>	Prominent Dashboard card: <code>**Prescriptive Learning Target**</code> with 1-click action routing.

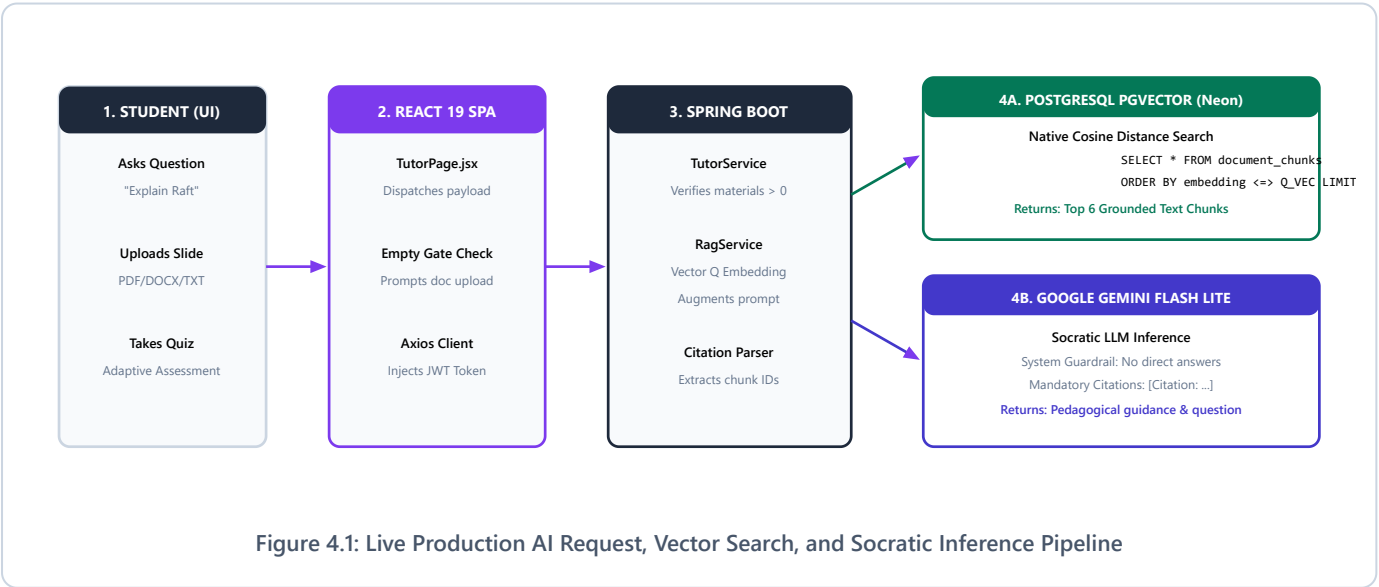


Figure 4.1: Live Production AI Request, Vector Search, and Socratic Inference Pipeline

5. Product AI Feature Details

5.1 Socratic AI Tutor

- **User Input:** The student inputs an inquiry or concept question in the tutor chat interface.
- **Request Flow:** Dispatched via `POST /api/projects/{id}/tutor/chat` with the question text.
- **Backend Construction:** `TutorService` checks for uploaded course materials. If zero documents exist, it throws `NO_MATERIALS` and halts inference. If documents exist, it converts the prompt into a 3072-dimensional vector via `RagService.getEmbedding(prompt)`, queries `document_chunks` using cosine distance (`<=>`), and retrieves the Top 6 matching excerpts.
- **Socratic Prompt Construction:**

```
You are an expert academic tutor. Context excerpts from verified course materials:
[Chunk 1]: ...
[Chunk 2]: ...
Student Question: {userPrompt}
Instructions:
1. Do not give the solution directly. Use Socratic dialogic questioning.
2. Formulate a targeted question or hint that leads the student toward discovery.
3. Explicitly cite chunks using: [Citation: filename (chunk #)].
```

- **Output Processing & Presentation:** The response is parsed for citation tags, persisted in `chat_messages`, and rendered in `TutorPage.jsx` with interactive citation badges that highlight the original document excerpts.

5.2 Adaptive Diagnostic Quiz Generation

- **User Input:** Student clicks `"Generate Practice Quiz"` inside a course project.
- **Processing Logic:** `QuizService` gathers representative semantic chunks across all project materials. It queries Gemini Flash Lite with a structured JSON schema prompt specifying question count, question type (Multiple Choice, True/False, Short Answer), options, correct answers, explanations, and target concept tags.
- **Validation & Persistence:** Backend validates that generated options contain valid keys and a non-empty correct answer, persisting them in `quizzes` and `quiz_questions` tables.
- **Student Presentation:** `QuizPage.jsx` provides an interactive assessment runner with per-question timers, option selection, immediate conceptual rationales, and grading breakdowns.

5.3 Concept Mastery Tracking & Prescriptive Recommendations

- **Scoring & Updates:** Upon quiz submission (`POST /api/quizzes/{id}/submit`), the backend computes percentage accuracy and updates the student's mastery score in `concept_mastery` using an exponential moving average:

```
New_Mastery = (Old_Mastery × 0.4) + (Attempt_Score × 0.6)
```

- **Targeted Recommendation Engine:** `MasteryService` identifies concepts with mastery score `< 70%`. If a weak concept exists, the Dashboard highlights it in the `**Targeted Mastery Recommendation Card**` (e.g. `**Concept Retention • Binary Search (35%)*`), routing the student directly to focused quiz practice or tutor questioning with a single click.

6. AI Models & Providers: Explicit Differentiation

To ensure complete transparency and prevent conflation between development tooling and production systems, the table below provides a strict, unambiguous matrix of all AI technologies associated with this project:

Model / System Name	Provider / Creator	Role Classification	Exact Deployment Context
Antigravity IDE Assistant	Google DeepMind	Development Tool (Category A)	Assisted developer during local coding; not accessible or present in the deployed application.
Antigravity Browser Subagent	Google DeepMind	Development Tool (Category A)	Autonomous browser testing during local verification; not part of production runtime.
gemini-flash-lite-latest	Google AI Studio	Production Model (Category B)	Embedded in live Spring Boot backend (<code>app.ai.gemini.text-model</code>); powers Socratic tutoring & quiz synthesis.
gemini-embedding-001	Google AI Studio	Production Model (Category B)	Embedded in live Spring Boot backend (<code>app.ai.gemini.embedding-model</code>); generates 3072d dense vectors.
PostgreSQL pgvector	PostgreSQL / pgvector	Production Vector Subsystem	Neon Serverless Database; calculates native cosine distance similarity for RAG chunk retrieval.

7. Responsible AI Usage & Governance

- **Human Code Auditing:** Every AI-suggested code snippet was reviewed for architectural soundness, memory efficiency, and security before being integrated into git.
- **Prevention of Hallucinations via Grounding:** The Socratic AI Tutor strictly references ingested documents. If a query falls outside the project's source materials, the system guardrail explicitly directs the model to decline answering rather than inventing facts.
- **Empty Materials Gate:** Prevents ungrounded inference by enforcing that a student must upload at least one verified course document before accessing tutoring or diagnostic quizzes.
- **Zero Secret Exposure:** No API keys, OAuth client secrets, or database passwords are hardcoded in git. All sensitive credentials are provided dynamically via environment variables (`GEMINI_API_KEY` , `JWT_SECRET`).
- **Pedagogical Safeguards:** Prompts enforce interactive questioning rather than homework-solving shortcuts, fostering genuine learning and cognitive retention.

8. Limitations & Lessons Learned

8.1 Technical Limitations Encountered

- **Cloud Firewall Restrictions on Outbound SMTP:** Render's free tier firewalls block outbound raw TCP traffic on ports 25, 465, and 587. Even with valid Gmail credentials, direct SMTP fails. This necessitated engineering an HTTP REST email transport via Resend (HTTPS port 443) and implementing dev-code auto-fill fallbacks.
- **React Hook Order Determinism (Error #310):** AI code generation initially placed a `useMemo` hook below an early `if (loading) return skeleton` check in `DashboardPage.jsx`. In production, this caused a fatal React crash on re-render. Resolving this reinforced the importance of human code review regarding React's strict Rules of Hooks.
- **Embedding Dimensions & Schema Alignment:** Google's `gemini-embedding-001` outputs 3072-dimensional vectors. Schema migrations had to explicitly align the PostgreSQL column type to `VECTOR(3072)`, requiring specialized Flyway DDL coordination.
- **Single-Origin OAuth Restrictions:** Google Identity Services strictly rejects origins not pre-registered in Google Cloud Console. Transitioning between preview URLs required developer intervention to authorize Vercel production domains.

8.2 Key Lessons Learned

- **AI Tools Require Technical Guidance:** AI assistants are highly capable of generating boilerplate and diagnosing errors, but cannot replace architectural judgment, domain modeling, or comprehensive verification.
- **RAG Quality Depends on Chunking Strategy:** A 400-token window with 50-token overlap provided the optimal balance between granular vector cosine similarity and contextual comprehension for academic text.
- **Stateless Architecture Ensures Seamless Deployment:** Decoupling the Vite/React frontend onto Vercel's edge network and the Spring Boot backend into Render's container runtime ensured zero coupling and clean independent scalability.

9. Summary

AI-Assisted Development (Category A):

The developer utilized Google Antigravity IDE and autonomous agentic subagents for boilerplate generation, Flyway migration drafting, React Error #310 debugging, and headless browser deployment verification. All generated outputs were reviewed, tested, and approved by the developer.

AI-Powered Product (Category B):

The deployed AiProf platform incorporates Google Gemini Flash Lite (`gemini-flash-lite-latest`) and Gemini Embeddings (`gemini-embedding-001`) paired with PostgreSQL `pgvector` to deliver grounded Socratic tutoring, 3072d vector search, adaptive diagnostic quiz synthesis, and real-time concept mastery tracking to students.